

TP 9 (programmation) - Cryptographie.

Comme d'habitude, vous devez rendre votre TP dans le délai indiqué. Un fichier `cesar.py` contenant les en-têtes des fonctions et des doctests est fourni, vous devez le compléter. N'oubliez pas que vous pouvez créer vos propres tests.

Exercice 1 (Code de César : cryptographie et cryptanalyse)

Le but de cet exercice est de réaliser un système permettant de chiffrer des messages, en utilisant le code de César : on part d'un message en clair et on change toutes les lettres en les décalant d'un même nombre de positions (ce nombre est appelé la *clé*). Pour décoder le message, il suffit alors de décaler ses lettres dans l'autre sens, à condition de connaître la clé. Pour cet exercice, nous vous encourageons à chercher dans la documentation quelles sont les fonctions qui peuvent vous être utiles. **Attention, ce qui suit ne fonctionne que pour des caractères sans accent !**

On va d'abord écrire la fonction d'encodage du texte : `chiffre(en_clair, cle)`, puis une fonction de décodage. Toutes les lettres sont chiffrées, mais pas les blancs ni la ponctuation. Par exemple :

```
>>> cache = chiffre('Toto est une girafe, avec des petites jambes.', 12)
>>> print(cache)
'Fafa qef gzq sudmrq, mhqo ppe bqfufqe vmynqe.'
>>> print(dechiffre(cache, 12))
'Toto est une girafe, avec des petites jambes.'
```

- (a) Écrivez une fonction `chiffre_lettre` qui prend une seule lettre en paramètre et renvoie la lettre chiffrée (attention à la ponctuation et aux majuscules). Pour décaler les lettres, vous aurez besoin de deux fonctions de base de Python : `ord` et `chr` (documentées ici : <http://docs.python.org/3/library/functions.html>). La fonction `ord` prend une chaîne contenant un seul caractère et renvoie son *code ASCII* (`ord('a')` donne 97, `ord('b')` 98, etc.). Et la fonction `chr` fait l'inverse : elle renvoie le caractère en fonction du code ASCII. Par exemple, le programme suivant permet de créer une liste contenant les lettres de l'alphabet (remarque : on peut obtenir le même résultat avec `string.ascii_lowercase` et `split`) :

```
alphabet = []
for i in range(26):
    alphabet.append(chr(ord('a') + i))
```

Vous devez calculer l'indice de la lettre dans l'alphabet, puis le décaler avec la clé (en prenant soin de rester entre 0 et 25 : par exemple, `'z'` décalé de 2 donne `'b'`) et retrouver le caractère correspondant. **Attention, le nombre 97 ne doit jamais apparaître dans votre code !** Pour vérifier : lorsque l'on fait

```
for c in alphabet: # initialisé cf. plus haut
    print(chiffre_lettre(c, 10), end=' '):
```

on doit obtenir : `k l m n o p q r s t u v w x y z a b c d e f g h i j`.

Ne passez pas trop de temps sur cette question : si vous êtes bloqué, appelez votre chargé de TP.

- (b) On constate que l'on fait deux fois le même travail, pour une majuscule ou une minuscule. Rajoutez une fonction auxiliaire qui prend `'a'` ou `'A'` en paramètre et permet de ne faire le calcul qu'une seule fois.

- (c) Enfin, écrivez la fonction `chiffre(texte)` qui renvoie le texte chiffré.
- (d) Passons au déchiffrement. Écrivez la fonction `dechiffre(texte_cache, cle)` qui permet de décoder (et renvoyer) un texte qui a été chiffré avec la clé. Soyez astucieux, ne copiez-collez pas votre code, réutilisez plutôt vos fonctions!

Exercice 2 (Code de César : cryptanalyse et améliorations)

- (a) Passons à la cryptanalyse, c'est-à-dire l'art de déchiffrer les messages sans avoir la clé ! Pour cela, nous allons essayer d'utiliser nos connaissances sur les fréquences des lettres dans un texte. En effet, vous n'êtes pas sans savoir que la lettre la plus fréquente en français est le 'e' ... Donc, pour savoir quelle clé a permis de coder le texte, une première approche naïve consiste à tester toutes les clés possibles et à identifier celle qui donne le texte contenant le plus de 'e'.

Écrivez la fonction `devine_cle(texte_cache)` qui renvoie la clé devinée en utilisant ce principe et testez-la sur l'exemple. Attention, cette technique fonctionne d'autant mieux que le texte est long. Essayez de retrouver le texte original dans les deux cas suivants.

Premier essai :

```
Tuqd, v'mu ymzs qzq bayyq. Mbdqe, v'mu qg ppe tmxxgouzmfuaze, bxquz
p'tmxxgouzmfuaze.
Vq hakmue ppe qxqbtmzfe, ppe xuoadzqe qf ppe bmzpm. Qf uxe qfmuqzf
daege, fage daege.
```

Deuxième essai :

```
Jli c'rsrkkrek ul mrjzkrj, le rezdrc rl kyfiro zeuzxf, r c'rzxlzccfe jrwire,
ez le trwriu, ez le tyriretfe, drzj gclfk le rikzjfe, j'rmretrz, kirzerek le
size u'rcwr.
Zc j'rggifyr, mflcrek c'rgcrkzi u'le tflg mzw, drzj c'rezdrc gize jfe mfc,
uzjgrirzjrek urej cr elzk rmrek hl'zc rzk gl c'rjrzcczi.
```

Si vous voulez savoir pourquoi ça ne fonctionne pas sur le second texte, cliquez là : [indice](#).

- (b) (**facultatif**) Cette façon de faire n'est pas très efficace, elle force à décoder 26 fois ... Il vaudrait mieux chercher la lettre la plus fréquente dans le texte et chercher la clé permettant de la faire correspondre avec un 'e'. Écrivez la fonction `devine_cle2` qui utilise ce principe.

Indice : une fois que vous connaissez la position `pos` dans l'alphabet (entre 0 et 25) de la lettre la plus fréquente, il suffit de calculer : $(pos + \text{ord}('a') - \text{ord}('e')) \% 26$.

- (c) On vient de voir qu'il n'est pas très difficile de déchiffrer le message, même sans la clé. Cette méthode de chiffrement n'est donc pas très sûre. Pour empêcher de deviner en utilisant les fréquences, on va chiffrer en décalant de 1 la clé de codage à chaque mot. Avec l'exemple de "Toto ...", ça donne : Fafa rfg ibs vxgput, qlus uvj hwlalwk ctfuxl.

Écrivez les fonctions `chiffre_mieux(texte, cle)` et `dechiffre_mieux(texte, cle)` qui utilisent ce principe.

- (d) On se rend compte que les fonctions `chiffre` et `chiffre_mieux` sont quasiment les mêmes. Transformez-les en une unique fonction `chiffre_encore_mieux` qui prend également en paramètre le décalage utilisé pour chaque mot : il suffit de mettre 0 pour retrouver `chiffre` et 1 pour `chiffre_mieux` :

```
>>> txt = 'Voici le texte'
>>> print(chiffre(txt, 12) )
Hauou xq fqjfq
>>> print(chiffre_encore_mieux(txt, 12, 0))
Hauou xq fqjfq
>>> print(chiffre_mieux(txt, 12) )
Hauou yr hslhs
>>> print(chiffre_encore_mieux(txt, 12, 1))
Hauou yr hslhs
```

Pensez à utiliser cette fonction pour déchiffrer aussi.